Neural networks fundamentals

Blank slide, confirmed — slides 5 and 6 are empty. Let me check slide 9's images closely and slide 14's diagram.
Course: LING 285: Human Language and Technology
Title of Lecture: Neural Networks 1
Date: 9/22/2026
Professor: Khalil Iskarous

Section 1: Differential Equations as the Mathematical Expression of Dynamical Systems

- **Differential equations describe how systems change over time, and Newton first communicated this idea through an act of secrecy rather than open publication.** In 1676, Newton wrote a letter to Leibniz in which he expressed his idea about these equations as an anagram, which is a cryptographic device that scrambles the letters of a statement so that the content is hidden until the sender later reveals the correct arrangement. This method was the way that scientists of that era established priority over a discovery without giving away the actual content of the idea before they had fully proven it.
    - The anagram Newton sent was written as the character string **6accdae13eff7i3l9n4o4qrr4s8t12ux**, which encodes the letters of his Latin sentence along with how many times each letter appears.
    - When unscrambled, the Latin sentence reads **Data aequatione quotcunque fluentes quantitates involvente fluxiones invenire, et vice versa**.
- **The mathematician Vladimir Arnold later paraphrased the meaning of Newton's Latin sentence in a single, general statement: the laws of nature are expressed by differential equations.** This paraphrase captures the core idea that differential equations are not just a mathematical tool but the actual language in which physical and natural processes are written.
- **Inference and learning in large language models are themselves differential equations.** This connects the seventeenth-century mathematics of Newton directly to the mathematical operations underlying modern neural network systems, showing that the same class of equations used to describe physical dynamical systems also describes how these computational language models process information and update themselves.

Section 2: The Viscous Spring and Euler's Method

- **A viscous spring exhibits negative, or balancing, feedback, and this behavior is written as the differential equation shown below.**

$$\dot{x} = -\frac{1}{3}x$$

- **The symbol $\dot{x}$ simply means the change in x.** It represents how much the quantity x is increasing or decreasing at a given moment, rather than the value of x itself.
- **The purpose of a differential equation together with an initial condition is to determine the value of x at every point in time, and this complete solution is called the solution to the differential equation.** The initial condition tells us the starting value of x, and the equation tells us the rule by which x changes from that starting point onward.
    - This kind of solution corresponds to real phenomena in the world, including stochastic gradient descent, the cooling or warming of flesh, and the temperature of a room approaching its surroundings.
- **Working through this equation by hand shows exactly how the solution is built up step by step.** The initial condition is set so that x equals 100 at time $t = 0$.
    - The equation states that the change in x is equal to negative one-third of its current value.
    - Negative one-third of 100 is approximately -33.
    - Since the change in x is -33, and we started at 100, losing 33 brings the value of x to approximately 66.

- **This step-by-step process of updating a value based on its own current value is captured in a general recursive formula, shown below.**

$$x_t = x_{t-1} - \frac{1}{3}x_{t-1}$$

- **Applying this formula again starting from the new value of x continues the process.** Once x has become 66, the same rule is applied again: one-third of 66 is calculated and subtracted from 66 to produce the next value of x, and this process is repeated for each successive step in time.
    - A graph accompanies this process with x on the vertical axis and t on the horizontal axis, with the horizontal axis marked at the integer values 0 through 8 and beyond, showing that the solution is a curve that can be traced out point by point as time advances.
- **This entire procedure of repeatedly plugging the current value back into the equation to generate the next value is called Euler's Method.** Differential equations are solved to predict what the world will look like at future times using nothing more than repeated arithmetic.
- **This calculation was then carried out in Python**, meaning the same repeated arithmetic operation of subtracting one-third of the current value from itself was implemented as a computer program rather than performed by hand.

Section 3: Software 1.0

- **One photograph shows a young woman standing beside a stack of printed paper roughly as tall as she is, which represents the written program code of a large software system from an earlier era of computing.** This image illustrates what software looked like before neural networks: a human being wrote out, line by line, every instruction a computer needed to follow, and the resulting program could be printed out as a towering stack of physical pages.
- **A second photograph shows the same woman later in life, being awarded a medal by a president of the United States in a formal ceremony, with another well-known technology figure seated nearby among the other honorees.** Placing these two photographs together under the heading Software 1.0 marks a contrast between this earlier style of programming, in which every rule of the software was explicitly written by a human programmer, and the neural network approach that the rest of the material builds toward, in which the system's behavior comes from learned numerical weights rather than hand-written instructions.

[CONTINUE]

Section 4: A Neural Network — What It Does

- **A neural network is introduced by describing two tasks it can perform rather than by defining it directly.** In the first task, you are given many pictures of faces, each containing 48 million pixels, and you are asked to identify which one of 1000 possible people is shown in a given picture. In the second task, you are given speech waves and asked to identify, for each wave, which of 25 possible sounds that wave corresponds to.
    - The question of what kind of system can perform both of these tasks is answered with a single name: a Neural Network, abbreviated NN.
    - Rather than explaining why it is called a neural network, the approach taken here is to teach what a neural network is by describing what it does, in the same way you might understand an animal by observing how it walks and how it swims rather than by first defining it biologically.
- **The job of a neural network is to take in a large collection of numbers and return a probability distribution over the possible answers.** These input numbers could be the 48 million pixel values of an image or the numerical values that make up a sound wave, and the output is a distribution that assigns a likelihood to each possible identity or category.
    - For the speech task, this probability distribution is spread across the 25 possible sounds.

    - As a simplified example, suppose there are only 5 possible vowels instead of 25 sounds. A neural network might output the probability distribution shown below.

$$[.2,\ .2,\ .4,\ .2,\ 0]$$

- **This list of five numbers is a valid probability distribution because the values sum to 1, meaning 100 percent.** If we agree that the order of these five values corresponds to the vowels in the words read, rad, rude, Rud, and rod, then each number tells us how confident the system is that the sound belongs to that particular vowel category.
    - The first value of .2 means there is a 20 percent chance the sound is the vowel in read.
    - The second value of .2 means there is a 20 percent chance the sound is the vowel in rad.
    - The third value of .4 means there is a 40 percent chance the sound is the vowel in rude.
    - The fourth value of .2 means there is a 20 percent chance the sound is the vowel in Rud.
    - The fifth value of 0 means there is a 0 percent chance the sound is the vowel in rod.

Section 5: How a Neural Network Learns Through Stochastic Gradient Descent

- **The input to a neural network is described as a bunch of numbers, a signal, or a vector, such as the example shown below, and this input usually contains thousands of numbers.**

$$[4,\ 1,\ -3,\ 2,\ 1,\ .4,\ 1,\ 2,\ -3,\ \ldots]$$

- If these numbers represent a speech wave, they correspond to the pressure values of that wave measured over time.
- **The output of the neural network is a decision in the form of a probability distribution, meaning that given the input signal, the network produces a probability for each possible object it could correspond to**, such as the probability of each of the vowels read, rad, rude, Rud, and rod.
- **The mechanism by which a neural network turns an input signal into this output decision depends on knowledge stored in a large collection of weights, referred to as the w's.** The input signal vector is combined with these weights through many calculations, and a decision about the identity of the object is eventually produced, even though there can be several trillion weights and billions of calculations involved in reaching that decision.
    - These weights are what allow the network to make decisions, but the question of how the network comes to know the correct values for these weights still needs to be answered.
- **The process by which a neural network discovers the correct weight values is described with the phrase fake it till you make it, and the specific procedure used is called stochastic gradient descent, or SGD.** The network begins with weights that are completely random and improves them gradually through repeated exposure to signals.
    - The network is first given a signal, for example the vowel sound in Rud, without yet knowing the correct weights.
    - **Forward propagation** is the step in which the network uses its current, still poorly tuned weights to process the input signal and produce an output probability distribution, such as the example shown below, which does not reflect the correct answer.

$$[.4,\ .1,\ .2,\ .2,\ .1]$$

- The network is then told that this output was wrong, and that the correct answer should have been the distribution shown below, which places all of the probability on the Rud category.

$$[0,\ 0,\ 0,\ 1,\ 0]$$

- Comparing its incorrect output to this correct answer allows the network to recognize that it has made a large error, also called a loss.
- The network then adjusts its weights in a way that reduces this loss, meaning its next attempt at the same kind of signal will produce an output closer to the correct answer.
- This entire cycle of receiving a signal, producing an output, comparing it to the correct answer, measuring the loss, and adjusting the weights is repeated billions of times using many different signals.
- The remaining question is what specific calculations are actually performed on the input signal to produce these outputs, which is addressed by the theories of perception discussed next.

Section 6: Two Theories of Perception That Inform Neural Network Design

- **The stated goal of this material is to understand how a neural network uses its weights to process an input and arrive at the correct decision at the output.** For example, when the network is given a speech signal corresponding to the sound [z], meaning a waveform or pressure waveform, the correct decision produced at the output should be [z], and not [s], not [a], or any other incorrect sound.
- **Understanding how humans perceive speech, faces, and other stimuli is useful for understanding how neural networks are designed, because our own biological neural networks perform this same kind of perception, and this biological process can guide how we understand the artificial computation performed by NNs.**
- **Two basic theories of animal perception inform the design of neural networks: Templatic Perception and Hierarchical Perception.** These two theories are examined individually before being brought together to explain how neural networks actually function.

Section 7: Templatic Perception

- **Templatic perception is presented as a theory of how face perception works.** According to this theory, the retina in the eye first receives light information coming from a face that is being viewed.
    - The brain is understood to store a template, essentially a stored picture, for every face that a person already knows.
    - The brain then compares the incoming face, as received through the retina, against each of these stored templates.
    - Whichever stored template has the highest similarity to the incoming face is the one that is perceived as the identity of that face.
    - If a name is associated with the winning template, that name becomes the final perceptual decision, meaning the person recognizes the face and recalls the corresponding name.
- **This theory implies that the knowledge weights of a neural network function like perceptual templates.** Recall that a neural network combines its current weights with the input face pixels to produce a computed output, and that this output is wrong whenever the network has not yet learned the correct weights.
    - The way that a neural network's weights, or knowledge, or templates, are combined with the input is by determining the similarity between the weights and the input.
    - This idea is acknowledged as difficult to fully grasp at this point, but it is meant to be kept in mind as the material continues.

Section 8: Hierarchical Perception

- **Hierarchical perception is the theory that perception occurs in a sequence of stages rather than in a single step, a conclusion that psychologists and neuroscientists have recognized for a long time.** Under this theory, an input does not jump directly to a final decision. Instead, the input first leads to one new representation, that representation is then transformed into another representation, and this continues through additional stages until the final output decision is produced.

- **A labeled diagram of a primate brain accompanies this theory, showing the visual processing pathways that carry information through the brain in stages.** Beginning from the region labeled V1, the pathway proceeds through V2, then V3 and V4, and continues into a region labeled MT and MST.
    - From these areas, one branch of connections proceeds toward the region labeled TEO and then into the region labeled IT, following the pathway on the underside of the brain toward the temporal lobe, with this branch marked separately from the other branch of the diagram.
    - Another branch of connections proceeds from the V1 through V4 and MT and MST region upward into the parietal areas labeled MIP, VIP, LIP, and AIP, and continues from there into the frontal areas labeled FEF, PMv, PMd, and PF, with this upper branch explicitly labeled as the dorsal pathway.
    - The arrows connecting these labeled regions run in multiple directions between neighboring areas, illustrating that information passes from one processing stage to the next and that each stage produces a new representation of the input before passing it forward, consistent with the hierarchical staging described in the accompanying text.

Section 9: Neural Networks as Templatic Hierarchical Perceivers

- **Neural networks are described as combining both of the theories already discussed, functioning as templatic hierarchical perceivers, beginning with the templatic aspect of this combination.** The key idea behind templatic perception is that some internal template made of numbers is compared against an input made of numbers in order to determine how similar the template is to the input.
- **Neural networks operationalize this idea by assuming that the input is a list, or vector, of numbers, and that the internal template weights are also a list, or vector, of numbers.** This raises the specific question of how two lists of numbers can be compared to determine whether they are similar to each other.
- **This comparison is illustrated with a specific numerical example involving one input list and two candidate template lists, shown below.**

$$\text{Input} = [-1,\ 3,\ -2]$$

$$\text{Temp1} = [-2,\ 0,\ 3]$$

$$\text{Temp2} = [-2,\ 2,\ -1]$$

- The question posed with this example is whether the Input list is more similar to Temp1 or to Temp2.
- **The tool used to answer this kind of question is called the inner product, and it is computed in two steps.** First, the two lists being compared are multiplied together component by component, meaning each entry in one list is multiplied by the entry in the same position in the other list. Second, all of the resulting products are added together to produce a single number.
    - If the resulting inner product value is near 0, the two lists being compared are considered dissimilar.
    - If the resulting inner product value is large and positive, the two lists being compared are considered similar.

Section 10: Inference and Learning in Neural Networks

- **Inference in a neural network is the process in which the system is given a signal vector, and its weights are compared to that signal through the inner product, or template matching, operation described above, with this comparison repeated over and over again as the signal passes through successive stages, consistent with hierarchical perception.** This repeated, stage-by-stage application of the same underlying calculation is compared directly to Euler's algorithm, in which a value is repeatedly updated by applying the same rule to the result of the previous step.
- **Learning in a neural network is the process in which the system is given a signal whose correct identity is already known, and it is asked to infer that identity while starting from initial weights that are random.** Because the weights start out random,

the network is unable to produce the correct identity at first, and this is precisely why the weights must then be improved through stochastic gradient descent.